# App ReNotes

## Proyecto práctico: Notas de investigación

**Asignatura:** Programación para Dispositivos Móviles
**Tecnología base:** Flutter para Android
**Confeccionado por:** Ing. Roilán Rodríguez Castillo
**Entrega:** Viernes 09/10/2026 antes de las 23:59

# Contexto

En este proyecto, el estudiante debe construir una aplicación Flutter llamada **ReNotes** (Notas de Investigación). La app permite organizar el trabajo de investigación en proyectos, escribir notas largas dentro de cada proyecto y clasificarlas con etiquetas que el propio usuario crea y reutiliza. La aplicación no requiere servidor, base de datos, almacenamiento local ni autenticación. Todos los datos se mantienen en memoria mientras la app está en ejecución y se cargan desde datos de prueba incluidos en el proyecto al iniciarla.

El diseño visual es libre. El estudiante decide colores, tipografía, íconos, distribución y estilo general.

# Alcance del proyecto

La app debe funcionar en Android y estar construida en Flutter. Debe incluir seis pantallas, navegación básica con BottomNavigationBar, datos en memoria, gestión dinámica de etiquetas y autocompletado de etiquetas al escribir. No se aceptan manejadores de estado más allá de los que vienen en el SDK de Flutter. No se aceptan paquetes de almacenamiento local ni bases de datos.

# Modelo de datos

La app debe manejar al menos estas entidades:

| Entidad | Campos mínimos |
|---|---|
| Proyecto | Identificador único, nombre, descripción corta, fecha de creación, fecha de última modificación. |
| Nota | Identificador único, identificador del proyecto, título, descripción (texto largo), fecha de creación, fecha de última modificación, lista de etiquetas. |
| Etiqueta | Identificador único, nombre, color opcional. |

Reglas del modelo:

- Un proyecto tiene cero o muchas notas. Una nota pertenece a un solo proyecto.
- Una nota tiene cero o muchas etiquetas. Una etiqueta puede estar en muchas notas de cualquier proyecto (relación muchos a muchos).
- Las etiquetas son globales. No pertenecen a ningún proyecto.
- El nombre de una etiqueta es único. La comparación ignora mayúsculas, minúsculas, acentos y espacios al inicio o al final. `Metodología`, `metodologia` y `METODOLOGIA` son la misma etiqueta.
- El título de la nota es obligatorio. La descripción puede quedar vacía, pero debe aceptar textos de varios miles de caracteres sin perder contenido.

# Requisitos funcionales

## RF1. Pantalla Proyectos

La pantalla de proyectos debe mostrar:

- Lista vertical de proyectos de investigación.
- Nombre, descripción corta, cantidad de notas y fecha de última modificación de cada proyecto.
- Botón para crear un proyecto nuevo (nombre obligatorio).
- Opción para editar el nombre y la descripción de un proyecto.
- Opción para eliminar un proyecto, con confirmación. Al eliminarlo se eliminan sus notas. Las etiquetas no se eliminan.
- Mensaje de lista vacía cuando no existan proyectos.
- Acceso a la pantalla Notas del proyecto al tocar un proyecto.
- Uso obligatorio de `ListView.builder`.

## RF2. Pantalla Notas del proyecto

La pantalla de notas de un proyecto debe mostrar:

- Nombre del proyecto y cantidad de notas.
- Lista vertical de las notas del proyecto con título, un extracto de la descripción, etiquetas y fecha de modificación.
- Búsqueda por título o contenido dentro del proyecto.
- Filtro por una o varias etiquetas, usando solo las etiquetas presentes en las notas del proyecto.
- Botón para crear una nota nueva en este proyecto.
- Opción para eliminar una nota, con confirmación.
- Acceso a la pantalla Detalle de nota al tocar una nota, y al Editor desde el detalle.
- Uso obligatorio de `ListView.builder` y uso de `ListView.separated` en al menos una vista.
- Mensaje de lista vacía cuando el proyecto no tenga notas o el filtro no devuelva resultados.

## RF3. Pantalla Editor de nota

La pantalla del editor se usa para crear y para editar una nota. Debe mostrar:

- Campo de título (obligatorio, con validación y mensaje de error).
- Campo de descripción de texto largo, multilínea, con scroll interno y sin overflow.
- Campo de etiquetas con chips de las etiquetas ya asignadas y opción para quitar cada una.
- Botón para guardar y botón para cancelar.
- Aviso de confirmación si el usuario intenta salir con cambios sin guardar.
- Layout que se adapte al teclado abierto y a pantallas pequeñas sin overflow.

Autocompletado de etiquetas (requisito principal de esta pantalla):

- Mientras el usuario escribe en el campo de etiquetas, la app muestra las etiquetas existentes cuyo nombre coincida con el texto escrito. La coincidencia ignora mayúsculas, minúsculas y acentos, y busca el texto en cualquier parte del nombre.
- Las sugerencias incluyen etiquetas creadas en cualquier nota de cualquier proyecto.
- No se sugieren las etiquetas que la nota ya tiene asignadas.
- Al tocar una sugerencia, la etiqueta se asigna a la nota y el campo se limpia.
- Si el texto escrito no coincide exactamente con una etiqueta existente, la lista ofrece una opción para crearla con ese nombre y asignarla en el mismo paso.

- También se puede confirmar con la tecla de acción del teclado (Enter) o al escribir una coma.
- Está prohibido crear etiquetas duplicadas. Si el nombre ya existe, se reutiliza la existente.
- Se debe usar el widget `Autocomplete` o `RawAutocomplete` del SDK de Flutter.

## RF4. Pantalla Detalle de nota

La pantalla de detalle de nota debe mostrar:

- Título, proyecto al que pertenece, fecha de creación y fecha de última modificación.
- Descripción completa en un área con scroll.
- Etiquetas de la nota. Al tocar una etiqueta se abre la pantalla Etiquetas filtrada por esa etiqueta, mostrando las notas de todos los proyectos que la comparten.
- Botón para editar la nota.
- Botón para eliminar la nota, con confirmación.
- Botón para volver al listado anterior.
- Layout adaptable a pantallas pequeñas sin overflow.

## RF5. Pantalla Etiquetas

La pantalla de etiquetas permite gestionar las etiquetas y agrupar notas por ellas. Debe mostrar:

- Lista de todas las etiquetas existentes con la cantidad de notas que usa cada una.
- Botón para crear una etiqueta nueva de forma independiente de cualquier nota.
- Opción para renombrar una etiqueta. El cambio se refleja de inmediato en todas las notas que la usan. Si el nuevo nombre ya existe, la app lo rechaza con un mensaje claro.
- Opción para eliminar una etiqueta, con confirmación que indique cuántas notas se verán afectadas. Al eliminarla se quita de todas las notas, pero las notas no se borran.
- Búsqueda de etiquetas por nombre.
- Al tocar una etiqueta, lista de las notas que la tienen, de todos los proyectos, indicando el proyecto de cada nota.
- Posibilidad de seleccionar varias etiquetas para agrupar notas. El usuario elige si ver las notas que tienen todas las etiquetas elegidas o las que tienen al menos una.
- Acceso a Detalle de nota desde cada nota listada.
- Uso obligatorio de `ListView.builder`.

- Mensaje de lista vacía cuando no existan etiquetas.

### RF6. Pantalla Buscar

La pantalla de búsqueda global debe mostrar:

- Campo de búsqueda por título o contenido en las notas de todos los proyectos.
- Filtro opcional por proyecto y por etiquetas.
- Resultados con título, proyecto, etiquetas y extracto de la descripción.
- Resultados en una lista construida con `ListView.separated`.
- Acceso a Detalle de nota al tocar un resultado.
- Mensaje cuando no haya resultados.
- Actualización inmediata de los resultados después de crear, editar o eliminar notas o etiquetas.

### RF7. Navegación básica

La app debe incluir `BottomNavigationBar` con tres destinos principales: Proyectos, Etiquetas y Buscar. Desde ellos se debe poder llegar a las demás pantallas:

- Proyectos -> Notas del proyecto.
- Notas del proyecto -> Detalle de nota.
- Notas del proyecto -> Editor de nota (crear).
- Detalle de nota -> Editor de nota (editar).
- Detalle de nota -> Etiquetas filtrada por una etiqueta.
- Etiquetas -> Detalle de nota.
- Buscar -> Detalle de nota.
- Detalle de nota o Editor -> pantalla anterior (según corresponda).

Se puede usar `Navigator.push`, `Navigator.pushNamed` o `go_router`. La implementación debe ser coherente en toda la app (solo se puede usar 1, nunca ambos).

Al cambiar de pestaña en el BottomNavigationBar, cada destino debe conservar su estado (texto de búsqueda, filtros y posición de scroll).

## Estado y datos en memoria

- Todos los proyectos, notas y etiquetas viven en memoria mientras la app está abierta. No se exige que los datos sobrevivan al cierre de la app.
- Al iniciar, la app carga los datos de prueba incluidos en el proyecto.

- Los cambios en proyectos, notas o etiquetas deben verse en todas las pantallas sin recargar manualmente.

### Modelo de estado recomendado

La app debe mantener al menos:

- Lista de proyectos.
- Lista de notas.
- Lista de etiquetas.
- Relación entre notas y etiquetas.
- Texto de búsqueda y filtros activos (proyecto, etiquetas, modo todas o al menos una).
- Identificador del proyecto abierto.
- Identificador de la nota abierta en Detalle.
- Borrador de la nota en edición.

## Requisitos de scroll

### ListView

- Usar `ListView()` solo para pocos elementos conocidos.
- Usar `ListView.builder` para listas largas o de tamaño variable.
- Usar `ListView.separated` cuando haya divisores visuales entre elementos.
- Definir `itemCount` siempre que la cantidad de elementos sea conocida.
- Usar `SingleChildScrollView` o un campo con scroll propio para el texto largo del Editor y del Detalle, evitando overflow.
- Usar `Wrap` para mostrar las etiquetas de una nota, de modo que pasen a la línea siguiente cuando no caben.

## Datos de prueba

El proyecto debe incluir datos de prueba cargados en memoria con al menos:

- 3 proyectos de investigación.
- 15 notas repartidas entre los proyectos, con al menos 4 notas por proyecto.
- 8 etiquetas.
- Al menos 3 etiquetas usadas en notas de proyectos diferentes, para poder comprobar la reutilización y la agrupación entre proyectos.

- Al menos 5 notas con descripción de más de 600 caracteres, para comprobar el texto largo.
- Al menos 1 nota sin etiquetas y 1 etiqueta sin ninguna nota.

# Entregables

Cada estudiante debe entregar:

1. Proyecto Flutter completo y compilable, publicado en un repositorio de GitHub.
2. Capturas de pantalla de las seis pantallas, más una captura del autocompletado de etiquetas en funcionamiento, incluidas dentro del repositorio.
3. Archivo README en el repositorio que explique cómo inicializar y ejecutar el proyecto.

# Criterios de aceptación

- La app inicia sin errores en Android.
- No hay overflow visible en las pantallas principales, tampoco con el teclado abierto.
- Se puede crear, editar y eliminar proyectos, notas y etiquetas.
- Una nota guarda un título y una descripción de texto largo sin perder contenido mientras la app está abierta.
- El autocompletado sugiere etiquetas existentes mientras se escribe, sin distinguir mayúsculas ni acentos, y ofrece crear la etiqueta si no existe.
- No se pueden crear dos etiquetas con el mismo nombre.
- Una misma etiqueta se puede asignar a notas de proyectos diferentes.
- Renombrar o eliminar una etiqueta se refleja en todas las notas que la usan.
- La pantalla Etiquetas agrupa las notas de todos los proyectos por una o varias etiquetas.
- La búsqueda y los filtros devuelven resultados correctos y se actualizan al cambiar los datos.
- Se usa un solo método de navegación y ningún manejador de estado externo al SDK.
- El repositorio de GitHub contiene el código, las capturas de pantalla y el README.
- El código es legible y mantiene separación entre datos, estado, pantallas y widgets reutilizables.